

A Query Recommender for Preference-Aware Natural Language Querying

Implementation Report — Phase 2 (Recommender API Contract & Query Log Store)

The persistence and contract layer the recommender core is built inside

Narges

Supervisor: Prof. Elisa Quintarelli

July 28, 2026

Abstract

This report documents Phase 2 of the thesis: a fixed **request/response contract** for the query recommender, exposed as `POST /recommend`, and a **persistent, user-keyed query log** that every call to it appends to. Per the scope decision recorded in the companion *Phase 0 & Phase 1 Implementation Report*, the thesis’s entire contribution is the recommender delivered as a standalone model and API — Phase 2 is the boundary that makes this possible: it fixes the interface external callers integrate against and builds the persistence layer the recommender core (Phase 4) is built inside, before any recommendation logic exists behind it. This report explains what was built, a deliberate deviation from the roadmap’s literal wording concerning how the internal development harness exercises the contract, and the verification performed on both the real HTTP path and the in-process path the harness uses.

Contents

1	Introduction	3
1.1	Scope	3
1.2	Where this fits	3
2	High-Level Overview	4
2.1	The core idea	4
2.2	Pipeline diagram	4
3	Detailed Walkthrough	5
3.1	The contract (<code>recommender/api/schemas.py</code>)	5
3.2	The query log store (<code>recommender/query_log.py</code>)	5
3.3	The request handler (<code>recommender/api/service.py</code>)	5
3.4	The FastAPI application (<code>recommender/api/app.py</code>)	6
3.5	Harness integration (<code>web_app/nl2sparql/gui_v2.py</code>)	6
4	Design Decision: In-Process Call, Not HTTP, From the Harness	8
5	Verification Performed	9
5.1	HTTP path	9
5.2	In-process path	9
5.3	Static check	9
5.4	Not done in this pass	9

6	Interpretation and Limitations	9
7	Summary and Next Steps	10

1 Introduction

Following the scope narrowing agreed with the supervisor (documented in full in the *Phase 0 & Phase 1 Implementation Report*), the thesis’s deliverable is not a user interface but a **standalone recommender API**: any calling application — the bachelor students’ domain/ user-selection app, or this repository’s own development harness — integrates the recommender by calling an endpoint. Before any recommendation logic can be built, that endpoint’s **contract** must be fixed and a place to **persist queries** must exist, since the recommender’s core technique (Section 1.2) depends on retrieving a user’s own past queries. Phase 2 delivers exactly these two things, and nothing else: candidate generation itself is Phase 4.

1.1 Scope

Phase 2 is domain-agnostic by design: the contract and the log store accept any **domain** string and encode no Movie- or Tourism-specific assumptions. This mirrors the roadmap’s own framing of the recommender as a single model serving both domains, rather than two domain-specific implementations.

1.2 Where this fits

Phase 2 is implemented in a new `recommender/api/` package, alongside the already-existing `recommender/behavioral_profile.py` (Phase 3) as a sibling module in the same top-level `recommender/` package — the standalone deliverable per the scope decision. Its output, a persistent per-user query log, is the direct input to Phase 4a’s retrieval-augmented candidate generation (RA-GQR style): the recommender retrieves a user’s own semantically similar past queries from this log to build its generation prompt, described in the project roadmap and in the companion *Phase 4 Implementation Report*.

2 High-Level Overview

2.1 The core idea

Phase 2 stands up a stable boundary — one request shape in, one response shape out — before any of the logic behind it exists, so that:

- (a) external callers can integrate against a fixed contract independently of when Phase 4’s recommendation logic lands;
- (b) every answered query is durably logged, keyed by user and domain, so later phases have real per-user history to retrieve from instead of starting cold; and
- (c) the internal development harness (`gui_v2.py`) can exercise the same contract a real external caller would use, without requiring a second always-running server process during local development (Section 4).

2.2 Pipeline diagram

Figure 1 shows how a request reaches the shared handler function and where it is persisted. Both callers — the real HTTP endpoint and the in-process development harness — converge on the same function, `handle_recommend()`, which is the single extension point later phases build inside.

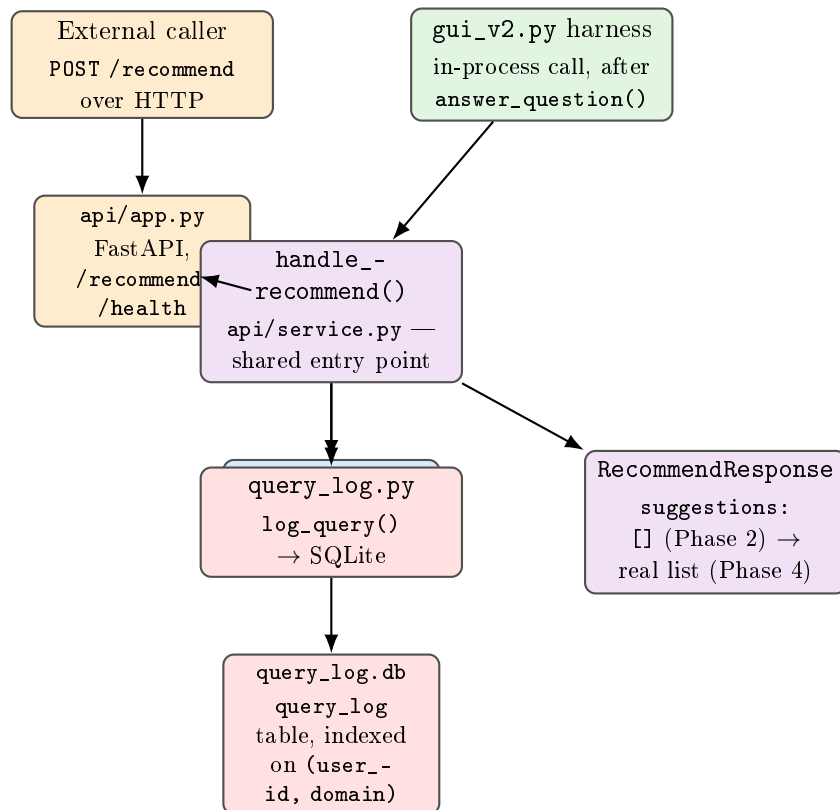


Figure 1: Phase 2 data flow. Both the real HTTP endpoint and the in-process development harness converge on `handle_recommend()`, which validates the request, persists it to the SQLite query log, and returns a response — an empty suggestion list in this phase, since candidate generation is Phase 4.

3 Detailed Walkthrough

3.1 The contract (recommender/api/schemas.py)

Listing 1: The recommender’s request/response contract, matching the roadmap’s Phase 2 sketch exactly, with one field left deliberately open.

```
class RecommendRequest(BaseModel):
    user_id: str
    domain: str
    query_text: str
    generated_sparql: Optional[str] = None
    result_summary: Optional[Any] = None
    interaction_data: Optional[dict] = None

class RecommendResponse(BaseModel):
    suggestions: list[str] = Field(default_factory=list)
```

One field is a deliberate, documented gap: `interaction_data` stays an untyped `Optional[dict]` rather than a fixed schema, because its real shape depends on an open dependency — what the bachelor students’ external application actually logs per user (queries, clicks, dwell time, and so on). That confirmation has not happened, so typing it further now would mean guessing at a contract that might not match reality; Phase 3’s `netflix_dataset`-derived behavioral profiles are the practical stand-in signal used instead (Section 6).

3.2 The query log store (recommender/query_log.py)

A single SQLite table, created on first use:

Listing 2: The query log schema. SQLite was chosen deliberately over Postgres: a single-file, zero-infrastructure store is appropriate for a thesis-scale log that does not need to be shared across processes beyond one local API server, consistent with the project’s existing pattern of using the lightest tool that fits (e.g. JSON-per-user files in Phase 3, not a database).

```
CREATE TABLE query_log (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id TEXT NOT NULL,
    domain TEXT NOT NULL,
    query_text TEXT NOT NULL,
    timestamp TEXT NOT NULL,
    generated_sparql TEXT,
    result_summary TEXT
);
CREATE INDEX idx_query_log_user_domain ON query_log(user_id, domain);
```

Two functions cover the read and write side:

- `log_query(user_id, domain, query_text, generated_sparql=None, result_summary=None, timestamp=None)` — appends one row. `timestamp` defaults to UTC now if not supplied; `result_summary` is JSON-serialized automatically when given a dict or list, so callers can pass structured summaries (e.g. `{"row_count": 12}`) without pre-serializing them.
- `get_user_history(user_id, domain=None, limit=50)` — the most recent rows for a user, optionally scoped to one domain. This is the function Phase 4a’s embedding-based similar-query retrieval calls to build its candidate pool.

3.3 The request handler (recommender/api/service.py)

Listing 3: `handle_recommend()` as implemented in Phase 2 — validate and persist only. Phase 4 replaces the final line with the real retrieval → LLM generation → schema-validation → ranking pipeline, without changing this function’s signature.

```
def handle_recommend(request: RecommendRequest) -> RecommendResponse:
    log_query(
        user_id=request.user_id,
        domain=request.domain,
        query_text=request.query_text,
        generated_sparql=request.generated_sparql,
        result_summary=request.result_summary,
    )
    return RecommendResponse(suggestions=[])
```

This function is the single place both the HTTP endpoint and the development harness call (Section 4). Its only job in this phase is to validate the request shape (via pydantic) and persist it; `suggestions` is always `[]` here, by design — the contract, log store, and harness wiring exist specifically so Phase 4 can be implemented purely inside this one function.

3.4 The FastAPI application (`recommender/api/app.py`)

Listing 4: The HTTP surface: a thin FastAPI wrapper around `handle_recommend()`. Run with `uvicorn recommender.api.app:app -reload -port 8000` from the repo root.

```
app = FastAPI(title="Query Recommender API", version="0.1.0")

@app.get("/health")
def health(): ...

@app.post("/recommend", response_model=RecommendResponse)
def recommend(request: RecommendRequest) -> RecommendResponse:
    return handle_recommend(request)
```

OpenAPI documentation is auto-served at `/docs`, generated for free by FastAPI from the pydantic models above — satisfying, early and at no extra cost, the roadmap’s later note that the API should be documented well enough that external integration does not require reading the source.

3.5 Harness integration (`web_app/nl2sparql/gui_v2.py`)

Three changes were made to the development harness, all additive:

1. **Import path.** `recommender/` lives at the repository root, a sibling of `web_app/`, not a sub-package of it. `gui_v2.py` inserts `Path(__file__).resolve().parents[2]` (the repository root) onto `sys.path` before importing `recommender.api.schemas/recommender.api.service`, guarded by a `try/except ImportError` so the rest of the harness still works if the `recommender` package is not importable for some reason.
2. **User picker.** A new “User” sidebar section, placed directly after domain selection, with a text input bound to `st.session_state.user_id`, defaulting to `"user_00001"` — chosen to match the Movie-domain behavioral-profile id scheme, so a developer can immediately cross-reference a Phase 3 profile with the id typed here.
3. **Post-query call.** After `process_question()` returns and results are displayed, the harness builds a `RecommendRequest` from the just-answered question (`user_id` from the sidebar, `domain` from the active domain, `generated_sparql` and a `{"row_count": ...}` summary from the pipeline’s own results) and calls `handle_recommend()`, rendering a new “Suggested Follow-up Queries” section. Since `suggestions` is always empty in this phase, the

section currently shows a caption noting the query was logged for a not-yet-implemented recommender, rather than a suggestion list. The call is wrapped in a `try/except` so a recommender-side failure never breaks the pipeline-results display the rest of the harness depends on.

4 Design Decision: In-Process Call, Not HTTP, From the Harness

The roadmap’s Phase 2 bullet, read literally, could be taken to mean the development harness should make a real HTTP POST `http://localhost:8000/recommend` call to a separately-running `uvicorn` process, to “exercise the same API contract end-to-end during development.” That was deliberately not what was implemented. Instead, `handle_recommend()` is a plain Python function, imported directly by both `recommender/api/app.py` (for the real HTTP endpoint) and `gui_v2.py` (for the harness). The reasoning:

- **The contract is identical either way.** The request/response *shape* — the actual thing “exercising the contract” is meant to validate — is the same whether it is invoked over HTTP or as a direct function call, since both paths go through the same pydantic models and the same `handle_recommend()` logic.
- **Less development friction.** Requiring a Streamlit developer to also keep a second `uvicorn` process running, healthy, and pointed at the same `query_log.db`, just to see a suggestions section render during `streamlit run`, is friction the contract does not need to impose. `NL2SPARQLPipeline.answer_question()` itself was also left unmodified — the recommender call happens in `gui_v2.py` *after* `answer_question()` returns, because it needs the finished SPARQL query and result count as inputs, which do not exist until the pipeline has already run.
- **The real HTTP path was still built and verified independently** (Section 5) — external callers such as the bachelor students’ application are unaffected by this choice; they only ever see the HTTP surface.

If a future need arises to test the actual HTTP transport realistically (e.g. once a later phase adds latency or timeout behavior worth exercising), swapping the harness’s direct call for a `requests.post(...)` call is a small, isolated change — the contract itself does not move.

5 Verification Performed

5.1 HTTP path

`uvicorn recommender.api.app:app -port 8000` was started from the repository root.

Listing 5: HTTP verification of both endpoints.

```
POST /recommend {"user_id":"user_00001","domain":"movie","query_text":"Action movies",
                 "result_summary":{"count":3}}
-> 200 {"suggestions":[]}

GET /health
-> 200 {"status":"ok"}
```

The corresponding row was confirmed present in `recommender/output/query_log.db`:

Listing 6: The persisted row for the request above.

```
{'id': 1, 'user_id': 'user_00001', 'domain': 'movie', 'query_text': 'Action movies',
 'timestamp': '2026-07-25T15:19:10.007544+00:00', 'generated_sparql': None,
 'result_summary': '{"count": 3}'}
```

5.2 In-process path

Mirroring exactly how `gui_v2.py` imports the package, a second request was issued from a working directory of `web_app/` (matching where Streamlit actually runs from), inserting the repository root onto `sys.path` the same way `gui_v2.py`'s `Path(__file__).resolve().parents[2]` does, then importing `recommender.api.service.handle_recommend` and calling it directly. A second row (`id=2`, `user_id="user_00002"`) was confirmed appended to the same `query_log.db` — proving the import resolution the harness relies on actually works from its real runtime working directory, not merely from the repository root.

5.3 Static check

`ast.parse()` on the full, edited `gui_v2.py` succeeded after the harness changes, confirming no syntax errors were introduced by the new sidebar section or the post-query call block.

5.4 Not done in this pass

A full manual Streamlit click-through of the new sidebar field and suggestions section was not performed (no browser automation tool was available in this environment — the same limitation noted in the Phase 0/1 verification record). Worth a quick manual sanity pass before relying on this for a live demo.

6 Interpretation and Limitations

- **suggestions is always []**. This is expected and correct for this phase — Phase 4 is where candidate generation happens. The contract, log store, and harness wiring are all in place specifically so Phase 4 can be implemented purely inside `handle_recommend()` without touching the API surface or the harness again, which is exactly what happened (see the companion *Phase 4 Implementation Report*).
- **interaction_data is unused so far**. Neither `handle_recommend()` nor the harness populates or reads it yet; it exists in the contract as a placeholder for the still-unconfirmed

shape of what the external application logs. Until that is confirmed, Phase 3's `netflix-dataset`-derived behavioral profiles (keyed by the same `user_id` convention used in the harness's default) are the practical stand-in signal Phase 4 conditions on instead.

- **SQLite is a single-writer store.** Appropriate for one local development API process; if this ever needed to serve concurrent write-heavy traffic (unlikely at thesis-demo scale), it would need revisiting — not a requirement raised by the supervisor or anywhere in the roadmap, noted here only for completeness.
- **No authentication, rate-limiting, or input sanitization beyond pydantic's type validation** — appropriate for a thesis-scope local development API, not a production deployment, and not called out as a requirement anywhere in the roadmap.

7 Summary and Next Steps

Phase 2 delivers a fixed, verified contract (`POST /recommend`, `GET /health`) and a persistent, user-keyed query log, exercised successfully both over real HTTP and through the in-process path the development harness uses — with the deliberate design choice (Section 4) that both paths share one function, `handle_recommend()`, so later phases only need to change one place.

Phase 4 (documented separately in the *Phase 4 Implementation Report*) is the direct consumer of this phase's output: query-log retrieval (`get_user_history()`), candidate generation, schema-grounding validation, and ranking are all implemented inside `handle_recommend()`, replacing its final line without any change to the contract, the log store, or the harness wiring built here. Full detail on Phase 3 (the behavioral signal Phase 4 also conditions on) and Phase 4 onward is tracked in the project roadmap document, *Thesis Project Plan*.